Servicios NLP expuestos por medio de API

Nombre: _____

INDICATIONS: Trabajo en los grupos de PBL
PUNTOS: 10

I. Objective

Your team will fine-tune a pretrained Transformer encoder for **extractive question answering** and deploy it as an **access-controlled web service** behind a fixed API. On presentation day, the instructor will call your API live with a hidden test set and evaluate the responses automatically.

This is an AI engineering project. You may use AI coding agents, but you are expected to know what your code does, what it does not do, and why.

II. Constraints

- **Duration:** 1.5 weeks.
- **Teams:** your PBL groups (4–5 students).
- **Models:** pretrained base encoder checkpoints only; you fine-tune them yourselves.
- **AI coding agents:** allowed (Claude Code, Gemini, Codex, etc.).
- **Language:** English.
- **Training hardware:** university GPUs when available, or your own hardware.
- **Serving hardware:** your own laptop, exposed through a public tunnel (Section IX). Plan for CPU-only inference.
- **Latency:** not graded, but a per-request timeout and a global time cap apply (Section XII).

III. Task

Extractive question answering with unanswerable questions (SQuAD 2.0 style). Given a context paragraph and a question, your service returns the answer as a span of the context, or no answer if the context does not contain it.

IV. Models

Choose one encoder and justify your choice.

Family	Examples	Notes
Classic encoder	DistilBERT, BERT-base, RoBERTa-base	Laptop-CPU friendly
Modern encoder	DeBERTa-v3-base, ModernBERT-base	State-of-the-art encoders

Decoders (GPT-style models) are not allowed for the implementation: they are slow on a laptop CPU, their generated answers do not fit exact-match evaluation, and the time frame does not allow doing both well. Decoders are covered in your report instead (Section XIII).

V. Rules

1. Start from a base checkpoint. Checkpoints already fine-tuned on SQuAD are **not allowed**. Training logs and loss curves are required.
2. No commercial API calls (OpenAI, Gemini, Anthropic, etc.). The model runs locally.
3. The service must implement the API contract (Section VII) exactly, including access control.
4. The access token must be read from an environment variable. It must never be written in code or committed to the repository.
5. Git history must be preserved. Do not squash it into a single commit.

Submissions are checked automatically for compliance with these rules.

VI. Data

- **Training:** SQuAD 2.0 train set (≈130k questions, ≈1/3 unanswerable). You may subsample.
- **Public sample set:** `public_sample.json`, 5 items in the `/predict` request format, with reference answers and their character offsets. Two are unanswerable and one has a context longer than 512 tokens. Released with this brief for local testing and the dry run.
- **Hidden test set:** 50 questions written by the instructor on texts not taken from SQuAD, ≈25 % unanswerable. It is not released.

VII. API contract

The full machine-readable contract is the OpenAPI file `qa_api_contract.yaml`, distributed with this brief. This section summarizes it. If the two disagree, the YAML file prevails.

VII.1. General rules

- **Base URL:** the public HTTPS URL of your tunnel (Section IX). Locally, the service listens on port `8000`.
- **Format:** all bodies are JSON, UTF-8, with header `Content-Type: application/json`.
- **Authentication:** protected endpoints require the header `Authorization: Bearer <token>`. The service reads the token from the environment variable `QA_API_TOKEN`. Comparison is exact and case-sensitive.
- **Errors:** every error response has this body:

```
{"error": "<code>", "detail": "<optional text>"}
```

  where `<code>` is `unauthorized`, `bad_request` or `internal_error`, and `detail` is optional.

- **Timeout:** the harness waits at most 5 s per request.
- **Extra fields** in your responses are allowed and ignored. A missing required field makes the response invalid.

Endpoint	Method	Auth	Success	Error codes
`/health`	GET	Public	200	503 while the model is loading
`/info`	GET	Protected	200	401
`/predict`	POST	Protected	200	400 or 422 (invalid body), 401, 500

VII.2. `GET /health` (public)

Reports whether the service can answer `/predict`. Returns `ok` only once the model is loaded.

```
curl http://<host>:8000/health

200 {"status": "ok"}
503 {"status": "loading"}
```

VII.3. `GET /info` (protected)

Field	Type	Required	Meaning
`team`	string	yes	Your team name as registered with the instructor (no spaces)
`model`	string	yes	Hugging Face id of the base checkpoint you fine-tuned from
`params`	integer	yes	Total parameters of the served model
`max_seq_length`	integer	no	Maximum tokens per model input window
`doc_stride`	integer	no	Token overlap between windows for long contexts

```
curl -H "Authorization: Bearer $QA_API_TOKEN" http://<host>:8000/info

200 {"team": "team-03", "model": "distilbert-base-uncased",
     "params": 66362880, "max_seq_length": 384, "doc_stride": 128}
401 {"error": "unauthorized"}
```

VII.4. `POST /predict` (protected)

Request body

Field	Type	Constraints
`id`	string	1–64 characters; echoed back unchanged
`context`	string	1–5000 characters, English; may exceed your model's `max_seq_length` in tokens
`question`	string	1–500 characters, English

Response body (200)

Field	Type	Meaning
`id`	string	Same value as the request
`answer`	string or `null`	Span copied verbatim from the context, or `null` for no answer
`start`	integer or `null`	0-based character offset of the answer in the context; `null` if and only if `answer` is `null`
`score`	number in $[0, 1]$	Confidence in the returned output (span or no-answer); not graded

The answer must satisfy `context[start : start + len(answer)] == answer`. A response that breaks this rule, has a different `id`, or misses a required field counts as a failed item.

Request body, saved as `request.json`:

```
{
  "id": "q-017",
  "context": "The Lempa River is the longest river in El Salvador, flowing about 422 km from
    Guatemala to the Pacific Ocean.",
  "question": "How long is the Lempa River?"
}
```

```
curl -X POST http://<host>:8000/predict \
  -H "Authorization: Bearer $QA_API_TOKEN" \
  -H "Content-Type: application/json" \
  -d @request.json

200 {"id": "q-017", "answer": "about 422 km", "start": 61, "score": 0.91}
```

Unanswerable example (same context, question "When was the first dam built on the Lempa River?"):

```
200 {"id": "q-018", "answer": null, "start": null, "score": 0.84}
```

Error examples:

```
401 {"error": "unauthorized"}
400 {"error": "bad_request", "detail": "field 'question' is required"}
500 {"error": "internal_error"}
```

What counts as a failed item

A non-200 response, malformed JSON, a missing required field, a mismatched `id`, an `answer` and `start` pair that does not match the context, or no response within 5 s.

VIII. Access control

- Your service validates the bearer token on `/info` and `/predict`, and returns `401` for a missing or invalid token. `/health` stays public.
- A **development token** is distributed with this brief, for development and the dry run.
- A **different token** is given on presentation day. You set it in `QA_API_TOKEN` before the evaluation starts.
- Access control is checked automatically at the dry run and on presentation day.

IX. Network setup: public tunnel

Your service runs on a team member's laptop. The instructor's harness reaches it through a **public HTTPS URL** created with a tunnel tool. This does not depend on the classroom Wi-Fi or on university network policies: the laptop only needs internet access.

The URL is public, so anyone on the internet can call your service. This is why access control (Section VIII) matters.

Allowed tools

Use one of these three. Install and test a second one as a backup.

Tool	Account	Public URL	Notes
ngrok	Free account	`https://<name>.ngrok-free.app` (one fixed domain per account)	Free plan: 20,000 HTTP requests and 1 GB per month. Browsers see a warning page; the harness is not affected.
Cloudflare Quick Tunnel	None	`https://<random>.trycloudflare.com`	The URL changes every time you restart the tunnel. For testing only, no uptime guarantee.
Tailscale Funnel	Free account	`https://<device>.<tailnet>.ts.net`	Requires MagicDNS and HTTPS enabled in your tailnet. A new URL can take up to 10 minutes to work. Funnel is in beta.

Commands

Your service listens on port 8000 on your laptop. Then run one of:

```
# ngrok (once: ngrok config add-authtoken <your ngrok authtoken>)
ngrok http 8000

# Cloudflare Quick Tunnel (no account needed)
cloudflared tunnel --url http://localhost:8000

# Tailscale Funnel (logged in to your tailnet)
tailscale funnel 8000
```

Each tool prints the public URL when it starts.

Procedure

1. Start your service and check it locally:

   ```
   curl http://localhost:8000/health
   ```

2. Start the tunnel and copy the public URL.
3. Run the curl tests below against the public URL, from a laptop or phone **on another network** (for example, mobile data).
4. Send the URL to the instructor before your dry-run slot and again before your presentation slot. If the URL changes (Cloudflare Quick Tunnel restarts, for example), send the new one.
5. Keep the laptop plugged in, with sleep disabled, and keep both the service and the tunnel running for the whole session.

Testing with curl

Run these five tests before the dry run. Replace `<public-url>` with your tunnel URL (or `http://localhost:8000` for a local test). The option `-i` prints the status line, so you see the HTTP status code and not only the body.

#	Test	Expected
1	`/health`, no token	`200 {"status": "ok"}`
2	`/info`, no token	`401`
3	`/info`, wrong token	`401`
4	`/info`, valid token	`200` with `team`, `model`, `params`
5	`/predict`, valid token	`200` with `id`, `answer`, `start`, `score`

For test 5, create `request.json` with the `id`, `context` and `question` of one item from `public_sample.json`, and compare the answer with that item's reference answers.

Linux and macOS (bash or zsh):

```
export QA_API_TOKEN=<your development token>

curl -i https://<public-url>/health
curl -i https://<public-url>/info
curl -i -H "Authorization: Bearer wrong" https://<public-url>/info
curl -i -H "Authorization: Bearer $QA_API_TOKEN" https://<public-url>/info
curl -i -X POST https://<public-url>/predict \
  -H "Authorization: Bearer $QA_API_TOKEN" \
  -H "Content-Type: application/json" \
  -d @request.json
```

Windows (PowerShell). Type `curl.exe`, not `curl`: in Windows PowerShell, `curl` is an alias of `Invoke-WebRequest`, which takes different options. `curl.exe` is included in Windows 10 and later. A backtick (`` ` ``) at the end of a line continues the command on the next line.

```
$env:QA_API_TOKEN = "<your development token>"

curl.exe -i https://<public-url>/health
curl.exe -i https://<public-url>/info
curl.exe -i -H "Authorization: Bearer wrong" https://<public-url>/info
curl.exe -i -H "Authorization: Bearer $env:QA_API_TOKEN" https://<public-url>/info
curl.exe -i -X POST https://<public-url>/predict `
  -H "Authorization: Bearer $env:QA_API_TOKEN" `
  -H "Content-Type: application/json" `
  -d "@request.json"
```

Sending the body from a file (`-d @request.json`) avoids JSON quoting problems in the shell. Never commit your token: set it in the terminal or in an untracked `.env` file.

Rules

- The tunnel points to the service running on your laptop. Cloud hosting is not allowed.
- The tunnel adds network latency to every request. Measure it at the dry run; the 5 s timeout applies to the full round trip.
- The dry run must be done through the tunnel, not on `localhost`.
- **Last resort:** if no tunnel works on presentation day, the instructor's harness runs on your laptop against `localhost`, with an encrypted test set whose key is given at that moment.

X. Timeline

Days	Work
1–2	Model choice, SQuAD 2.0 setup, baseline
3–5	Fine-tuning, no-answer threshold
6–7	API per contract (including access control), packaging (Docker or venv), local tests with the public sample set
8	**Dry run and code submission** (Section XI)
9–10	**Presentation:** live evaluation, pitch and Q&A

XI. Dry run and code submission (day 8)

- Your service must pass `/health`, the access-control checks, `/info` and the public sample set from the instructor's evaluation harness, **through your public tunnel URL**.
- The harness measures your time per item. Teams projected to exceed the global time cap are warned.
- Additional automated checks of the service are run. They are not graded.
- Push a git tag named `submission` and share the repository URL. **Only that commit is reviewed.** Private repositories must invite the instructor as a collaborator.
- Model weights stay out of git (use a release or a shared folder).
- Your code will be reviewed with AI assistance to prepare the presentation questions.

XII. Presentation day

Format

- 3 minutes of pitch, then 3 (± 1) minutes of Q&A.
- The live evaluation starts when your pitch starts and runs in parallel.

Live evaluation

- The harness sends the 50 hidden items one at a time.
- Per-request timeout: **5 s**. Global cap: **150 s**. Items not answered by the cap count as failed.
- Exact Match and F1 are shown live. Answers that partially overlap the reference are assessed by an automated LLM judge; that part of the score may be finalized after the session.

Network

Same as the dry run: the harness calls your public tunnel URL (Section IX). Send the URL to the instructor before your slot.

XIII. Deliverables

1. Code repository (training and service), tagged `submission`, history preserved, weights excluded.
2. Training logs and loss curves.
3. A running service implementing the API contract.
4. A short written report, `REPORT.md` in the root of the repository, in the commit tagged `submission`. Use the headings of `REPORT_TEMPLATE.md`, distributed with this brief. It includes:
   - Model choice and justification.
   - Training setup, loss curve and validation results.
   - How you chose the no-answer threshold.
   - Serving architecture and tunnel.
   - Results on the public sample set.
   - Model size and self-measured latency (not graded).
   - Problems encountered and how you fixed them.
   - **Decoder investigation:** how a decoder (GPT-style) or encoder–decoder architecture could address specific problems your team encountered, for example answers that are not a contiguous span, yes/no questions, or long contexts.
   - Known limitations.

   A report with a missing or empty section counts as not delivered.

XIV. Grading

Component	Points	Scope
Live evaluation	60	Team
Pitch	20	Team
Q&A	20	Individual
Total	**100**	

The lecture control is evaluated separately. The final grade is on a 0–100 scale, with no minimum: a student or team can earn 0.

Report condition. The report is not graded, but it is required. If it is missing, the final grade is halved:

$$\text{final} = (\text{live} + \text{pitch} + \text{qa}) \times \begin{cases} 1 & \text{report delivered} \\ 0.5 & \text{report missing} \end{cases}$$

XIV.1. Live evaluation (60 points, team)

Each of the 50 items receives a score; the **team score** is their average.

Case	Item score
Exact match, or correct `null` on an unanswerable item	1
Partial overlap, judged valid / partially valid / invalid	1 / 0.5 / 0
Failure: timeout, error, malformed response, or wrong `null`	0

The team score is mapped to points using two anchors:

- **Baseline:** a service that always answers `null`. Provisional value: **0.25**.
- **Reference:** a DistilBERT model fine-tuned by the instructor under the same constraints as yours, evaluated with the same harness. Provisional value: **0.70**.

$$\text{live} = \text{clip}\left(\frac{\text{team} - \text{baseline}}{\text{reference} - \text{baseline}},\ 0,\ 1\right) \times 60$$

Reaching the reference earns the full 60 points; exceeding it earns no extra. The anchor values will be confirmed on the final hidden test set before the presentations.

XIV.2. Pitch (20 points, team)

Three minutes, four criteria, each scored 0–5 (levels 1, 2 and 4 are intermediate).

Criterion	0	3	5
Model choice and justification	Missing	Model named, generic justification	Justified with a tradeoff specific to the task and to CPU serving
Training approach	Missing	Steps listed	Steps, key decisions (data subset, hyperparameters, no-answer threshold) and evidence (loss curve, validation)
Serving architecture	Missing	Framework named	Framework, how the model is loaded, how requests flow, how access control works
Decoder investigation	Missing	Generic decoder vs. encoder comparison	Tied to specific problems your team encountered

XIV.3. Q&A (20 points, individual)

Each student answers one question about the team's code, assigned at random on the spot.

Level	Description	Points
0	No answer, or wrong	0
1	Vague, generic, could apply to any repository	5
2	Right direction, but with errors or confusion	10
3	Correct, but misses key points or is not specific to your code	15
4	Correct, specific to your code, and covers the key points	20

- If your first answer scores level 0–2, you get **one** follow-up question, on the same scale, capped at level 3 (15 points). You keep the higher of the two scores. There are no further questions.
- The answer is individual. Teammates may not help.
- You may look at your own repository on screen. AI agents are not allowed during the Q&A.

XIV.4. Absence

A student absent on presentation day scores 0 on the whole project, unless the absence is justified.

XV. Readings

1. Vaswani et al., 2017. Attention Is All You Need.
2. Devlin et al., 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding (Sections 4.2–4.3, SQuAD fine-tuning).
3. Radford et al., 2018. Improving Language Understanding by Generative Pre-Training (GPT-1). Supports the decoder investigation.

4. Rajpurkar et al., 2018. Know What You Don't Know: Unanswerable Questions for SQuAD.
5. Hu et al., 2021. LoRA: Low-Rank Adaptation of Large Language Models (main body only). Optional; relevant to the decoder investigation.